

UNIVERSIDADE FEDERAL DO MARANHÃO
PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO (PPGCC)
MESTRADO EM CIÊNCIA DA COMPUTAÇÃO

Projeto de Pesquisa (Anteprojeto de Dissertação)

**Deteção eficiente de vulnerabilidades em código por modelos de
linguagem: um estudo sistemático de quantização e ajuste fino
eficiente sob avaliação descontaminada**

*Quão pequeno e barato pode ser um modelo mantendo desempenho na deteção de
vulnerabilidades?*

Mestrando(a): _____

Orientador(a): _____

Linha de pesquisa: Modelagem Computacional

São Luís, MA
2026

1. Introdução e contextualização

A detecção de vulnerabilidades em código-fonte é uma atividade crítica para a segurança de sistemas, tradicionalmente apoiada em análise estática, revisão manual e, mais recentemente, em aprendizado de máquina (Zhou et al., 2019). Com o avanço dos modelos de linguagem de grande porte (LLMs) para código, a tarefa passou a ser tratada também como classificação conduzida por esses modelos (Ding et al., 2024). O desafio prático é que os LLMs mais capazes possuem grande porte, exigindo muita memória de GPU e apresentando latência e custo de inferência elevados, o que limita seu uso em ambientes com recursos restritos.

Em paralelo, consolidaram-se técnicas de eficiência capazes de reduzir esse custo, com destaque para a quantização, que reduz a largura de bits dos pesos (Frantar et al., 2023; Lin et al., 2024), e para o ajuste fino eficiente em parâmetros, que permite especializar modelos com baixo custo de treinamento (Hu et al., 2022; Dettmers et al., 2023). Ao mesmo tempo, modelos abertos de porte pequeno e médio alcançaram desempenho competitivo (Lozhkov et al., 2024; Mishra et al., 2024; Hui et al., 2024). Surge, então, a pergunta que orienta este trabalho: quão pequeno e barato um modelo pode se tornar mantendo bom desempenho na detecção de vulnerabilidades, e em que medida o ajuste fino eficiente compensa a redução de tamanho e de precisão?

Essa pergunta ganhou urgência com uma constatação recente: boa parte do desempenho relatado na literatura não se sustenta quando a avaliação é rigorosa. Ao consolidar e depurar os dados, o benchmark PrimeVul mostrou que um modelo que atinge cerca de 68% de F1 em um conjunto tradicional despenca para valores próximos do acaso em condições realistas (Ding et al., 2024). Este projeto se insere na linha de Modelagem Computacional do PPGCC, no eixo de reconhecimento de padrões e aprendizado de máquina, e propõe um estudo empírico e reproduzível que una três frentes hoje tratadas de forma separada: eficiência (quantização e ajuste fino eficiente), detecção de vulnerabilidades e avaliação resistente à contaminação.

2. Justificativa

Grande parte dos resultados de detecção de vulnerabilidades apoia-se em conjuntos como Devign e Big-Vul (Zhou et al., 2019; Fan et al., 2020), cuja duplicação e cujos rótulos ruidosos podem inflar as métricas. O PrimeVul evidenciou o problema de forma marcante: um modelo que registra 68,26% de F1 no Big-Vul cai para 3,09% no conjunto depurado, e modelos proprietários avançados, mesmo com ajuste fino, apresentam desempenho próximo do palpite aleatório sob avaliação pareada (Ding et al., 2024). Estudos independentes reforçam a fragilidade da tarefa: pequenas alterações que preservam a semântica, como renomear variáveis, levam modelos avançados a errar em parcela

relevante dos casos (Ullah et al., 2024).

Ao mesmo tempo, a literatura de eficiência raramente reporta, de forma conjunta e padronizada, o efeito da quantização e do ajuste fino especificamente sobre essa tarefa, tampouco o custo associado (memória, latência, energia). Um estudo que una avaliação descontaminada, técnicas de eficiência e medição de custo oferece valor científico, por consolidar evidências rigorosas hoje dispersas, e valor prático, por orientar a escolha de modelos sob restrição de recursos.

3. Objetivos

3.1. Objetivo geral

Caracterizar de forma sistemática e reproduzível o compromisso entre eficiência e desempenho na detecção de vulnerabilidades em código por modelos de linguagem abertos, avaliando o efeito da quantização e do ajuste fino eficiente sob avaliação resistente à contaminação, e verificando se a especialização de modelos pequenos recupera o desempenho de modelos maiores.

3.2. Objetivos específicos

- Estabelecer baselines de detecção com modelos abertos pequenos e médios e com um codificador de referência (por exemplo UniXcoder), avaliando sobre um conjunto de dados depurado e pareado.
- Medir o efeito da quantização (8 e 4 bits) e comparar métodos distintos (por exemplo AWQ, GPTQ, NF4 e formatos GGUF) sobre a qualidade da detecção.
- Especializar os modelos pequenos por ajuste fino eficiente (QLoRA) com cabeça de classificação e comparar o modelo pequeno especializado com modelos maiores não especializados.
- Quantificar o efeito da contaminação, contrastando o desempenho em benchmarks legados com o desempenho em um conjunto rigoroso, deduplicado e pareado.
- Adotar métricas realistas (F1 da classe vulnerável, VD-S, acurácia pareada e taxa de falsos positivos) e medir o custo de cada configuração (memória, latência, vazão e energia), construindo curvas de trade-off e recomendações.
- Avaliar a robustez das configurações a perturbações que preservam a semântica, para verificar se o desempenho reflete compreensão e não atalhos textuais.

4. Revisão de literatura

Esta seção organiza o referencial teórico em quatro eixos (detecção de vulnerabilidades e sua avaliação por LLMs, contaminação e benchmarks, técnicas de eficiência e a lacuna de pesquisa) e reflete um levantamento recente da literatura de 2024 a 2026. Trata-se de uma versão a ser refinada com a leitura aprofundada de cada fonte.

4.1. Detecção de vulnerabilidades e sua avaliação por LLMs

A tarefa é hoje formulada em quatro granularidades: classificação binária de função, classificação por tipo de fraqueza (CWE), detecção em nível de linha ou instrução (localização) e detecção em nível de repositório, com raciocínio inter-procedural (Ding et al., 2024; Ahmed et al., 2025). Quanto às abordagens, a evolução vai das redes neurais em grafo, que modelam a semântica do programa, como o Devign (Zhou et al., 2019), aos modelos pré-treinados codificadores, como CodeBERT (Feng et al., 2020), GraphCodeBERT (Guo et al., 2021) e UniXcoder (Guo et al., 2022), e destes aos LLMs de código, como Qwen2.5-Coder, StarCoder2, DeepSeek-Coder e Granite Code. Uma observação recorrente é que codificadores totalmente ajustados, em especial o UniXcoder, permanecem baselines fortes, por vezes superando LLMs bem maiores na detecção (Yin et al., 2024).

As técnicas com LLMs incluem prompting sem exemplos e com poucos exemplos, raciocínio em cadeia, ajuste fino supervisionado, sistemas de agentes, recuperação aumentada por conhecimento e abordagens neuro-simbólicas que combinam o modelo com análise estática. Entre os resultados mais promissores, o IRIS acopla um LLM a análise estática e, em um conjunto de 120 vulnerabilidades em Java, detecta 55 com um modelo proprietário forte, contra 27 de uma ferramenta de análise estática consolidada, mantendo bom desempenho mesmo com um modelo aberto de 7B (Li et al., 2024). A recuperação de conhecimento também ajuda: o Vul-RAG eleva a acurácia pareada em cerca de 16 a 24% relativos (Du et al., 2024), embora um estudo de reprodutibilidade tenha observado um platô em torno de 0,30 de acurácia pareada, pouco acima do acaso, com modelos abertos (estudo de reprodutibilidade, 2026). No sentido oposto, evidências apontam fragilidade: modelos avançados mudam de resposta diante de renomeações e de pequenas adições ao código (Ullah et al., 2024), e há indícios de que modelos ajustados aprendem atalhos, associando domínios funcionais à probabilidade de vulnerabilidade em vez de compreender a causa-raiz (indício reportado em preprint recente de 2026).

4.2. Contaminação de dados e benchmarks

Os conjuntos clássicos apresentam limitações bem documentadas. O Devign tem rotulagem cuidadosa, porém cara, com parcela de rótulos imprecisos e duplicações; o Big-Vul, extraído de registros públicos de vulnerabilidades (CVE), é naturalmente desbalanceado e apresenta rótulos ruidosos; e o DiverseVul, embora mais amplo e diverso,

ainda herda ruído das fontes que consolida (Fan et al., 2020; Chen et al., 2023). Divisões padronizadas popularizadas por iniciativas como o CodeXGLUE herdaram esses problemas (Lu et al., 2021). O ponto de virada foi o PrimeVul, que melhora a rotulagem, deduplica, adota divisão temporal e propõe métricas realistas, revelando que o bom desempenho aparente em benchmarks legados não se sustenta em condições rigorosas (Ding et al., 2024). Duas contribuições metodológicas do PrimeVul são centrais para este projeto: a métrica VD-S, definida como a taxa de falsos negativos sob taxa de falsos positivos de no máximo 0,5%, e a avaliação pareada, na qual cada função vulnerável é comparada à sua versão corrigida, medindo se o modelo realmente distingue as duas.

Esse arranjo explica por que o F1 isolado engana. Em um teste balanceado entre funções vulneráveis e corrigidas, um classificador que responde sempre vulnerável obtém revocação máxima e F1 aparentemente razoável, ainda que sua taxa de falsos positivos seja total e sua utilidade prática nula. Por isso, reportam-se em conjunto o F1 da classe vulnerável, a taxa de falsos positivos, o VD-S e a acurácia pareada. Benchmarks recentes seguem essa direção de maior rigor, entre eles o MegaVul (Ni et al., 2024), o ReposVul, de nível de repositório (Wang et al., 2024), e propostas de 2025 com anotação em nível de instrução e verificação assistida por LLM (Ahmed et al., 2025; Li et al., 2024b). Reconhece-se ainda o risco de memorização dos conjuntos antigos pelos LLMs modernos, o que reforça a necessidade de divisões temporais e de dados posteriores ao corte de conhecimento dos modelos.

4.3. Técnicas de eficiência aplicadas à tarefa

Quantização. A quantização pós-treino reduz a memória e o custo de inferência. O GPTQ realiza a quantização com compensação de erro baseada em informação de segunda ordem (Frantar et al., 2023), enquanto o AWQ preserva pesos salientes identificados pela distribuição das ativações, com menor degradação em baixa precisão (Lin et al., 2024); o SmoothQuant trata a quantização conjunta de pesos e ativações (Xiao et al., 2023). Um ponto favorável a este projeto é que tarefas de classificação e compreensão tendem a ser bem mais robustas à baixa precisão do que a geração autorregressiva: evidências mostram degradação desprezível em modelos codificadores sob 4 bits, ao contrário do que ocorre com decodificadores voltados à geração (Wu et al., 2023). Como este trabalho trata a detecção como classificação, espera-se maior tolerância à quantização, hipótese a ser verificada empiricamente.

Ajuste fino eficiente. O LoRA treina matrizes de baixo posto mantendo os pesos congelados (Hu et al., 2022), e o QLoRA o combina à quantização em 4 bits, viabilizando, segundo seus autores, o ajuste fino de modelos grandes em uma única GPU sem perda relevante frente ao ajuste em 16 bits (Dettmers et al., 2023). Na tarefa em questão, trabalhos recentes indicam que o QLoRA com cabeça de classificação pode igualar codificadores

totalmente ajustados em conjuntos fáceis, alcançando F1 elevado no Big-Vul, mas apenas empatar com esses codificadores no PrimeVul, e que a formulação por classificação supera a formulação por geração (Ouchebara e Dupont, 2025). Resultados anteriores com LoRA em modelos de código também superaram baselines de codificadores em detecção binária, com número reduzido de parâmetros treináveis (Shestov et al., 2024).

Destilação. A destilação de conhecimento comprime modelos com pequena perda de eficácia, como no Avatar, que reduz codificadores de código em ordens de grandeza de tamanho e energia mantendo desempenho competitivo em predição de vulnerabilidade (Shi et al., 2023). Registra-se, porém, um trade-off entre compressão e robustez adversarial, relevante em contexto de segurança.

4.4. Lacuna de pesquisa

Apesar da riqueza de métodos, quase não há trabalho que meça, de forma isolada e sob avaliação descontaminada, o efeito da quantização sobre a qualidade de classificação de vulnerabilidade (e não sobre geração), combinado ao ajuste fino eficiente. Identifica-se, assim, a ausência de um estudo que (i) combine quantização e QLoRA em modelos pequenos; (ii) meça conjuntamente qualidade e custo; e (iii) empregue avaliação resistente à contaminação, contrastando explicitamente benchmarks legados e depurados e usando métricas pareadas. Preencher essa lacuna, verificando se a especialização de modelos pequenos recupera o desempenho de modelos maiores e qual método de quantização melhor preserva a qualidade na detecção, é o cerne deste trabalho.

5. Metodologia

O trabalho adotará abordagem experimental quantitativa, organizada em cinco etapas encadeadas, com registro sistemático para garantir reprodutibilidade.

- **Reprodução de baselines.** Selecionar modelos abertos pequenos e médios (por exemplo StarCoder2, Qwen2.5-Coder e Granite Code) e um codificador de referência (UniXcoder), avaliando em modo de classificação sobre um conjunto depurado e pareado, e registrando F1, taxa de falsos positivos, VD-S e acurácia pareada. Esta etapa consolida o arcabouço de avaliação.
- **Quantização.** Aplicar quantização em 8 e 4 bits com métodos distintos (AWQ, GPTQ, NF4 e formatos GGUF) e reavaliar a qualidade, isolando o efeito do método e da largura de bits sobre a detecção.
- **Ajuste fino eficiente.** Especializar os modelos pequenos por QLoRA com cabeça de classificação na partição de treino e reavaliar, comparando o modelo pequeno especializado com modelos maiores não especializados, para responder se a especialização recupera o desempenho.

- **Efeito da contaminação.** Contrastar o desempenho em benchmarks legados (Devign, Big-Vul) com o desempenho em conjunto rigoroso, deduplicado e pareado (PrimeVul), evidenciando eventual superestimação, e incluir um teste de robustez com perturbações que preservam a semântica.
- **Análise de trade-off.** Medir o custo de cada configuração (memória de GPU, latência de primeiro token e por token, vazão em tokens por segundo e energia), construir curvas de qualidade versus custo, identificar os pontos de melhor troca e derivar recomendações práticas por faixa de orçamento.

Como o F1 isolado é insuficiente em dados desbalanceados, a avaliação principal combinará F1 da classe vulnerável, VD-S, acurácia pareada e taxa de falsos positivos (Ding et al., 2024). O custo será medido em memória, latência, vazão e energia, esta última estimada a partir da potência da GPU amostrada por utilitários de telemetria. Como os experimentos serão conduzidos em GPUs alugadas sob demanda, o custo por hora da instância, multiplicado pelo tempo de execução, servirá de proxy reproduzível de custo total, complementando as demais métricas. As ferramentas previstas incluem a biblioteca Transformers e utilitários de ajuste fino eficiente e de quantização do ecossistema aberto, além de um servidor de inferência otimizado para as medições de latência e vazão.

6. Resultados esperados

Espera-se produzir: (i) uma caracterização quantitativa do compromisso entre qualidade e custo para as combinações de tamanho, precisão e método estudadas; (ii) evidência sobre se, e em que condições, um modelo pequeno especializado por QLoRA recupera o desempenho de um modelo maior; (iii) uma indicação de qual método de quantização melhor preserva a qualidade na detecção; e (iv) uma demonstração empírica do impacto da contaminação sobre as métricas reportadas. Vale registrar que um resultado igualmente relevante seria constatar que os modelos, mesmo especializados, permanecem próximos do acaso sob avaliação pareada, o que reforçaria a leitura de eficiência (obter desempenho equivalente a uma fração do custo). Como resultado secundário, pretende-se disponibilizar os scripts de avaliação para reprodutibilidade e submeter os achados a evento ou periódico da área.

7. Escopo e delimitação

O trabalho concentra-se na detecção de vulnerabilidades tratada como classificação, e nas alavancas de eficiência de quantização e de ajuste fino eficiente, com o tamanho do modelo como variável de estudo. Não fazem parte do escopo a proposição de novos algoritmos de quantização ou de treinamento, nem a localização fina da vulnerabilidade ou sua correção automática, que permanecem como extensões possíveis. Registra-se, por

transparência, que o autor conduz um trabalho de conclusão de curso de graduação em tema correlato, porém distinto, voltado à avaliação de eficiência na geração de código; os dois trabalhos compartilham fundamentação e infraestrutura, mas possuem tarefas, métodos e contribuições próprios.

8. Cronograma

Fase	Atividades	Meses
Estudo dirigido	Revisão do estado da arte, definição das métricas descontaminadas, montagem do arcabouço e reprodução de baselines (Etapa 1)	1 a 6
Consolidação	Quantização (Etapa 2), início do ajuste fino eficiente com cabeça de classificação (Etapa 3) e estudo de contaminação (Etapa 4)	6 a 12
Qualificação	Redação e defesa da qualificação com fundamentação, problema e resultados preliminares	12 a 15
Dissertação	Conclusão da Etapa 3, análise de trade-off (Etapa 5), redação da dissertação e submissão de artigo	15 a 24

O cronograma pressupõe um ciclo aproximado de vinte e quatro meses e será ajustado conforme o calendário do programa e a disponibilidade de recursos computacionais.

Referências

AHMED, S. et al. SecVulEval: Benchmarking LLMs for Real-World C/C++ Vulnerability Detection. Preprint, 2025. arXiv:2505.19828.

CHEN, Y.; DING, Z.; ALOWAIN, L.; CHEN, X.; WAGNER, D. DiverseVul: A New Vulnerable Source Code Dataset for Deep Learning Based Vulnerability Detection. In: RAID, 2023. arXiv:2304.00409.

DETTMERS, T.; PAGNONI, A.; HOLTZMAN, A.; ZETTLEMOYER, L. QLoRA: Efficient Finetuning of Quantized LLMs. In: NeurIPS, 2023. arXiv:2305.14314.

DING, Y. et al. Vulnerability Detection with Code Language Models: How Far Are We? (PrimeVul). In: ICSE, 2025. arXiv:2403.18624.

DU, X. et al. Vul-RAG: Enhancing LLM-based Vulnerability Detection via Knowledge-level RAG. 2024. arXiv:2406.11147; DOI:10.1145/3797277.

FAN, J.; LI, Y.; WANG, S.; NGUYEN, T. N. A C/C++ Code Vulnerability Dataset with Code Changes and CVE Summaries (Big-Vul). In: MSR, 2020, p. 508-512.

FENG, Z. et al. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In: Findings of EMNLP, 2020. arXiv:2002.08155.

FRANTAR, E.; ASHKBOOS, S.; HOEFLER, T.; ALISTARH, D. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. In: ICLR, 2023. arXiv:2210.17323.

GUO, D. et al. GraphCodeBERT: Pre-training Code Representations with Data Flow. In: ICLR, 2021. arXiv:2009.08366.

- GUO, D. et al. UniXcoder: Unified Cross-Modal Pre-training for Code Representation. In: ACL, 2022. arXiv:2203.03850.
- HU, E. et al. LoRA: Low-Rank Adaptation of Large Language Models. In: ICLR, 2022. arXiv:2106.09685.
- HUI, B. et al. Qwen2.5-Coder Technical Report. 2024. arXiv:2409.12186.
- LI, Z.; DUTTA, S.; NAIK, M. IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities. In: ICLR, 2025. arXiv:2405.17238.
- LIN, J. et al. AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. In: MLSys, 2024. arXiv:2306.00978.
- LOZHKOV, A. et al. StarCoder 2 and The Stack v2: The Next Generation. 2024. arXiv:2402.19173.
- LU, S. et al. CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation. In: NeurIPS Datasets and Benchmarks, 2021. arXiv:2102.04664.
- MISHRA, M. et al. Granite Code Models: A Family of Open Foundation Models for Code Intelligence. 2024. arXiv:2405.04324.
- NI, C. et al. MegaVul: A C/C++ Vulnerability Dataset with Comprehensive Code Representations. In: MSR, 2024. arXiv:2406.12415.
- OUCHEBARA, M.; DUPONT, G. Llama-based Source Code Vulnerability Detection: Prompt Engineering vs Fine Tuning. Preprint, 2025. arXiv:2512.09006.
- SHESTOV, A. et al. Finetuning Large Language Models for Vulnerability Detection. IEEE Access, 2025. arXiv:2401.17010; DOI:10.1109/ACCESS.2025.3546700.
- SHI, J. et al. Greening Large Language Models of Code (Avatar). Preprint, 2023. arXiv:2309.04076.
- ULLAH, S. et al. LLMs Cannot Reliably Identify and Reason About Security Vulnerabilities (SecLLMHolmes). In: IEEE S&P, 2024. arXiv:2312.12575.
- WANG, X. et al. ReposVul: A Repository-Level High-Quality Vulnerability Dataset. In: ICSE (Industry), 2024. DOI:10.1145/3639478.3647634.
- WU, X. et al. Understanding INT4 Quantization for Transformer Models. In: ICML, 2023. arXiv:2301.12017.
- XIAO, G. et al. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. In: ICML, 2023. arXiv:2211.10438.
- YIN, X.; NI, C.; WANG, S. Multitask-based Evaluation of Open-Source LLM on Software Vulnerability. IEEE Transactions on Software Engineering, 2024. arXiv:2404.02056.
- ZHOU, Y.; LIU, S.; SIOW, J.; DU, X.; LIU, Y. Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks. In: NeurIPS, 2019. arXiv:1909.03496.